Static Testing

All members will take each one of the three distinctions containing the MVC architecture: Model, View, Controller. Each one will use SonarQube and SpotBugs to complete Static Testing on the subpackages and classes inside their respective packages.

1.Static Testing-Controller package- Martin Vila

The first round of Static Testing was started with SonarQube as a good tool to find bugs. I pushed around 20 commits on different classes onto github to fix bugs that had: high, medium and low impact on the maintainability of our Software. The Controller is the package with the most classes out of the three and as such has around 20 classes, which makes its classes have high impacts on other ones. Trying to fix a bug would cause 8 related problems in other classes. The most usual bugs fixed were: Hard-coded credentials,Reliance on Platform Default Encoding, Incorrect null checks,etc.
The second round of Static Testing for the Controller package was conducted with SpotBugs, which helped fix the last bugs dispersed in different classes. In around 8 commits with SpotBugs the most usual bugs were: Dodgy code with static method , Regex with stack overflow danger on repetition and Impossible/ unsafe casts.
The last round saw SpotBugs and SonarQube take on a last analysis into all the classes of Controller, to check for any last missing bugs. There were only 3-4 bugs left without a proper solution.

2.Static Testing-Model package- Entea Bakiasi

The first run of Static Testing was conducted using SonarQube on the Model package, which helped identify bugs with high, medium, and low impact on maintainability. Multiple commits(21) were pushed to GitHub to address these issues across different model classes. Since the Model package represents the core business logic of the system, even small changes could affect several related classes. The most common issues fixed during this phase included exposure of internal representations through getter methods, high cognitive complexity, poor control-flow readability, redundant exception handling, and formatting and modifier-ordering problems. These fixes significantly improved encapsulation, readability, and long-term maintainability of the codebase.

The second run of Static Testing for the Model package was performed using SpotBugs, which helped detect deeper logical and structural issues that were not fully covered by SonarQube. Through several additional commits, common problems such as dodgy code patterns, unsafe or unnecessary casts, direct floating-point equality comparisons, and resource management warnings were addressed. In particular, SpotBugs highlighted cases where floating-point values were compared using `==`, which was corrected to avoid unreliable precision-based behavior. Encapsulation issues related to returning mutable collections were also resolved where possible.

In the final run, both SonarQube and SpotBugs were used to reanalyze the entire Model package to ensure no critical issues remained. At this stage, only a small number of low-severity warnings were left unresolved. The most notable remaining issue was related to the use of FileOutputStream, which was intentionally left unchanged due to its low risk and minimal impact on the overall functionality of the system. This issue did not affect correctness or stability and was considered acceptable within the project scope.

Overall, the static testing process greatly improved the quality, safety, and maintainability of the Model package, ensuring that the core logic of the application is robust and well-structured.